Chapter 10

Optimization in CSPs

Chapter 10. Optimization in CSPs

10.1 Introduction

10.2 The Constraint Satisfaction Optimization Problem

10.3 The Partial Constraint Satisfaction Problem

10.1 Introduction

Various scenarios:

- techniques for solving CSPs in which all solutions are equally good
- some solutions are better than others
- assignment of different values to same variable incurs different costs

The task is to find optimal solutions

- optimality is defined in terms of some application-specific functions

Constraint Satisfaction Optimization Problems (CSOP)

Constraints are so tight that one normally cannot satisfy all of them

- find compound labels which are as close to solutions as possible
- closeness may be defined in a number of ways

Partial Constraint Satisfaction Problems (PCSP).

10.2 The Constraint Satisfaction Optimization Problem

Definition 10.1:

A CSOP is defined as a CSP together with an optimization function f which maps every solution tuple to a numerical value:

$$(Z, D, C, f)$$

where (Z, D, C) is a CSP, and if S is the set of solution tuples of (Z, D, C), then

$$f: S \rightarrow \text{numerical value.}$$

Given a solution tuple T, we call $f(T)$ the f-value of T. ■

The task in a CSOP is to find the solution tuple with the optimal (minimal or maximal) f-value with regard to the application-dependent optimization function f.

In order to find the optimal solution, one potentially needs to find all the solutions first, and then compare their *f*-values. A part of the search space can only be pruned if one can prove that the optimal solution does not lie in it — which means either no solution exists in it (which involves knowledge about solutions) or that the *f*-value in any solution in the pruned search space is sub-optimal (which involves knowledge about the *f*-values).

Finding optimal solutions basically involves comparing all the solutions in a CSOP. Therefore, techniques for finding all solutions are more relevant to CSOP solving than techniques for finding single solutions.

We introduce two important methods for tackling CSOPs which have not been introduced in this book so far. They are the branch and bound (B&B) algorithm and genetic algorithms (GAs). The former uses heuristics to prune off search space, and the latter is a stochastic approach that has been shown to be effective in combinatorial problems.

Solving CSOPs with branch and bound

To apply the B&B to CSOP, one needs a heuristic function *h* which maps every compound label *CL* to a numerical value (*h*: *CL* → number). We call this value the *h*-value of the compound label. For the function *h* to be admissible, the *h*-value of any compound label *CL* must be an *over-estimation* (*under-estimation*) of the *f*-value of any solution tuple which projects to *CL* in a *maximization* (*minimization*) problem.

The Branch_and_Bound procedure below outlines the steps in applying a depthfirst branch and bound search strategy to solving the CSOP, where the maximum *f*value is required. Minimization problems can be handled as maximization problems by substituting all *f*- and *h*-value by their negation. For simplicity, this procedure returns only one solution tuple which has the optimal *f*-value; other solution tuples which have the same *f*-value are discarded.

```
PROCEDURE Branch_and_Bound( Z, D, C, f, h );
/* (Z, D, C) is a CSP; f is the function on solution tuples, the f-value is
    to be maximized; h is a heuristic estimation of the upper-bound of
    the f-value of compound labels */
BEGIN
    /* BOUND is a global variable, which stores the best f-value found
        so far; BEST_S_SO_FAR is also a global variable, which
        stores the best solution found so far */
    BOUND ← minus infinity; BEST_S_SO_FAR ← NIL;
    BNB( Z, { }, D, C, f, h );
    return(BEST_S_SO_FAR);
END /* of Branch_and_Bound */
```

```
PROCEDURE BNB(UNLABELLED, COMPOUND_LABEL, D, C, f, h);
BEGIN
    IF (UNLABELLED = { }) THEN
        BEGIN
            IF (f(COMPOUND_LABEL) > BOUND) THEN
                BEGIN            /* only one optimal solution is returned */
                    BOUND ← f(COMPOUND_LABEL);
                    BEST_S_SO_FAR ← COMPOUND_LABEL;
                END;
        END;
    ELSE IF (h(COMPOUND_LABEL) > BOUND) THEN
        BEGIN
            Pick any variable x from UNLABELLED;
            REPEAT
                Pick any value v from D_x;
```

```
            Delete v from D_x;
            IF (COMPOUND_LABEL + {<x,v>} violates no con-
               straints)
            THEN BNB(UNLABELLED – {x}, COMPOUND_LABEL
               + {<x,v>}, D, C, f, h);
         UNTIL (D_x = { });
      END /* of ELSE IF */
END /* of BNB */
```

Branch_and_Bound procedure is only sound and complete if $h(CL)$ indeed returns an upper-bound of the f-value. If the heuristic h may underestimate the f-value, then the procedure may prune off search space where optimal solutions lie, which causes sub-optimal solution tuples to be returned. A branch will be pruned if the h-value of the current node is lower than the bound (in a maximization problem). Even with the heuristic function fixed, B&B will prune off different proportion of the search space if the branches are ordered differently, because different bounds could be found under different branches.

Variables	Domains	Constraints
x_1	4, 5	
x_2	3, 5	$x_2 \neq x_1$
x_3	3, 5	$x_3 = x_2$
x_4	2, 3	$x_4 < x_3$
x_5	1, 3	$x_5 \neq x_4$

Task: to assign consistent values to the variables and maximize $\sum$(values assigned)

Figure 10.1 Example of a CSOP

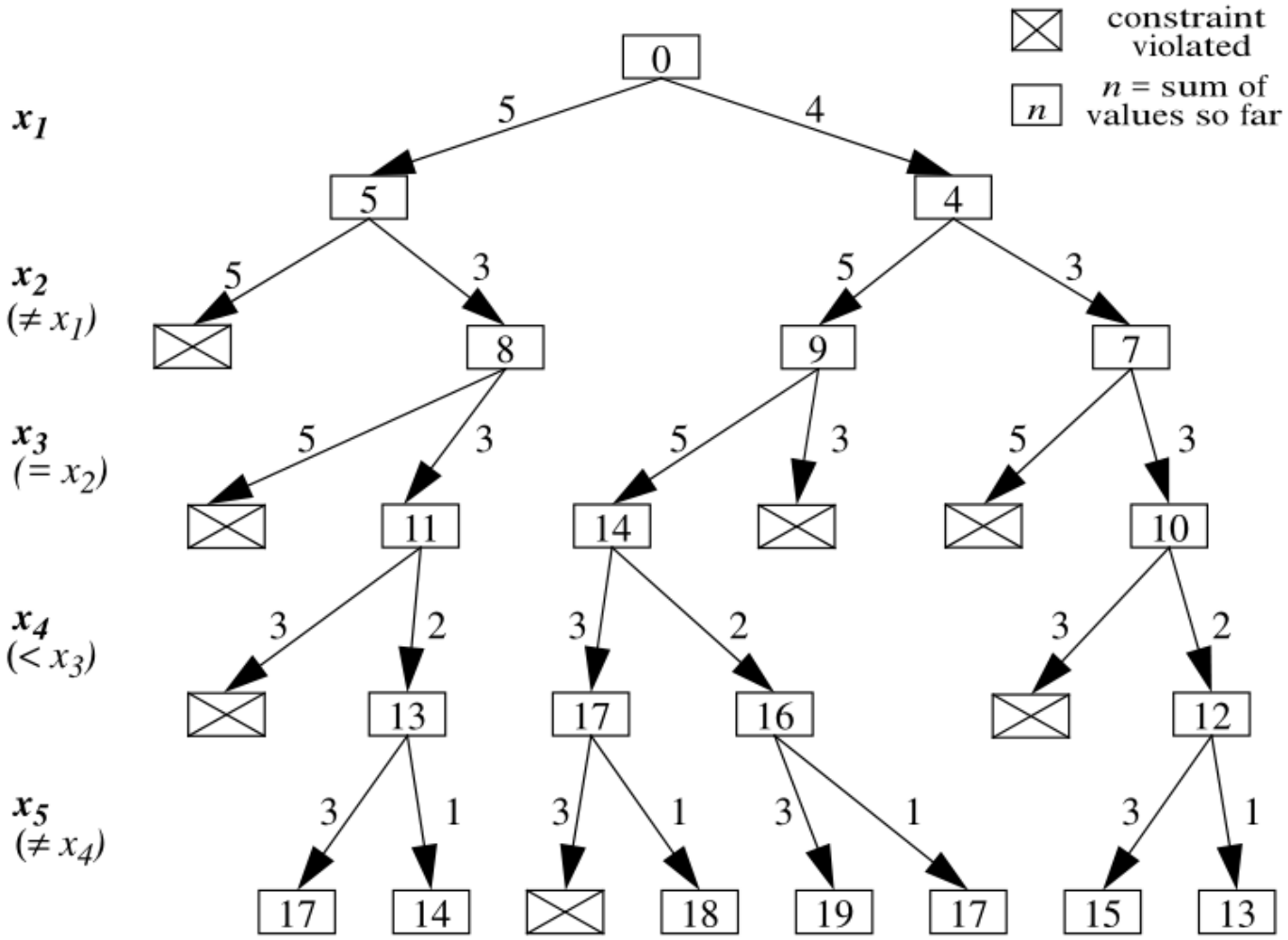


Figure 10.2 The space searched by simple backtracking in solving the CSOP in Figure 10.1 (branches which represent the assignment of greater values are searched first)

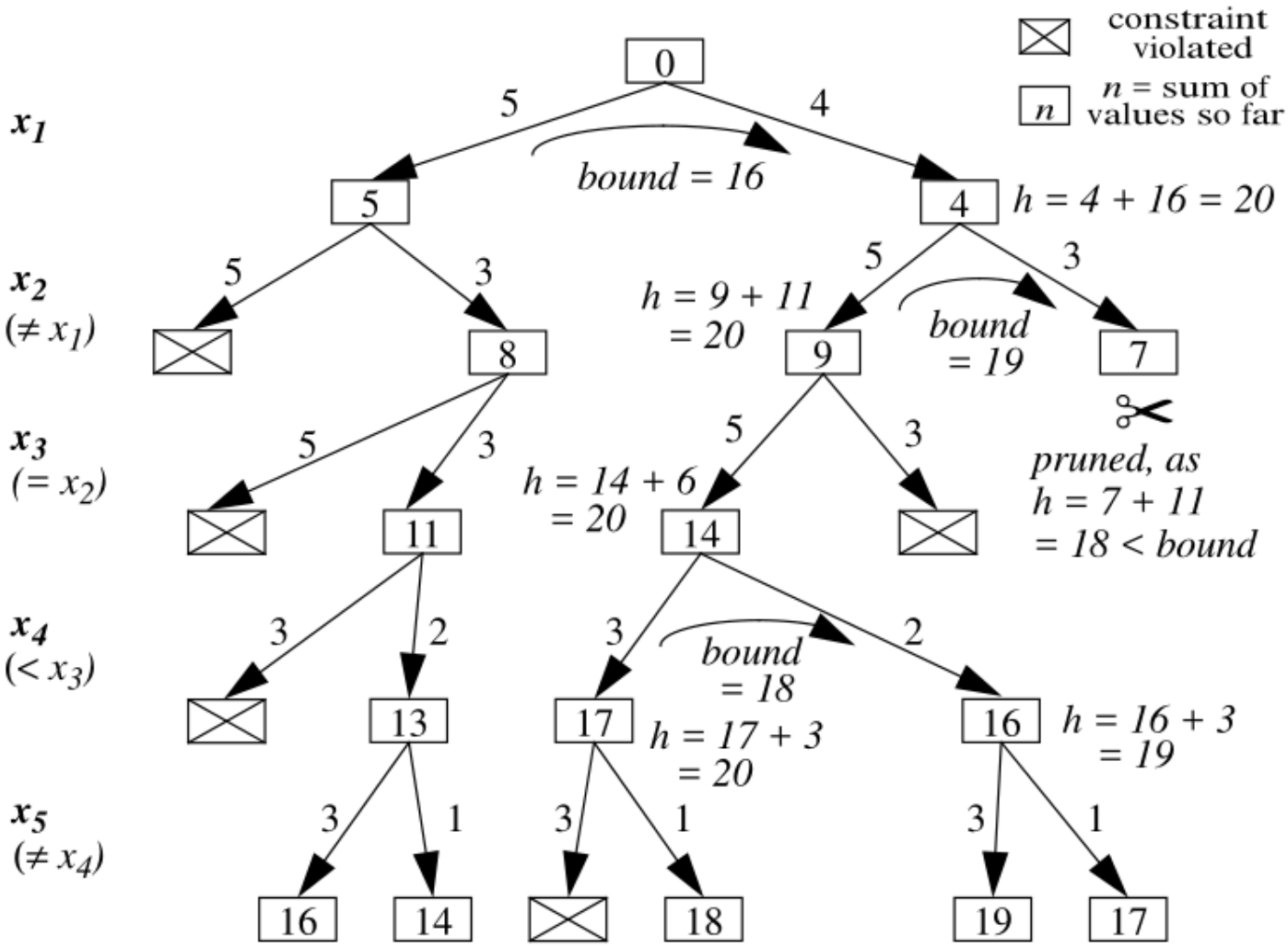


Figure 10.3 The space searched by Branch & Bound in solving the CSOP in Figure 10.1: branches which represent the assignment of greater values are searched first; $h(x)$ = value assigned + $\sum$(maximal values for the unlabelled variables)

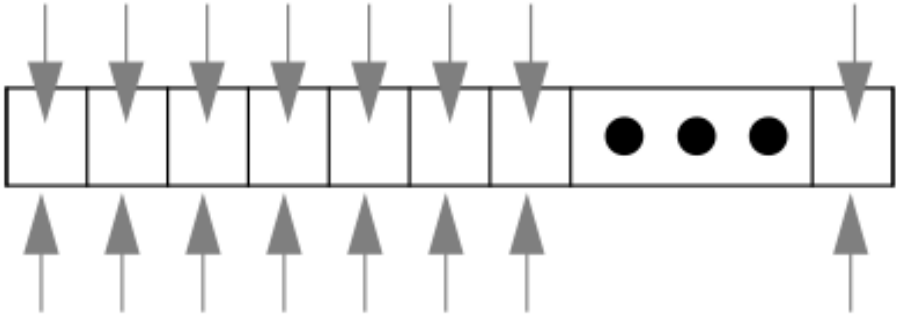
Tackling CSOPs using Genetic Algorithms

Genetic algorithms (GAs) are a class of stochastic search algorithms which borrow their ideas from evolution in nature. GAs have been demonstrated to be effective in a number of well known and extensively researched combinatorial optimization problems, including the travelling salesman problem (TSP), the quadratic assignment problem (QAP), and applications such as scheduling.

We describe the GAs, and evaluates their potential in solving CSOPs. Preliminary research has suggested that GAs could be useful for large but loosely constrained CSOPs where near-optimal solutions are acceptable.

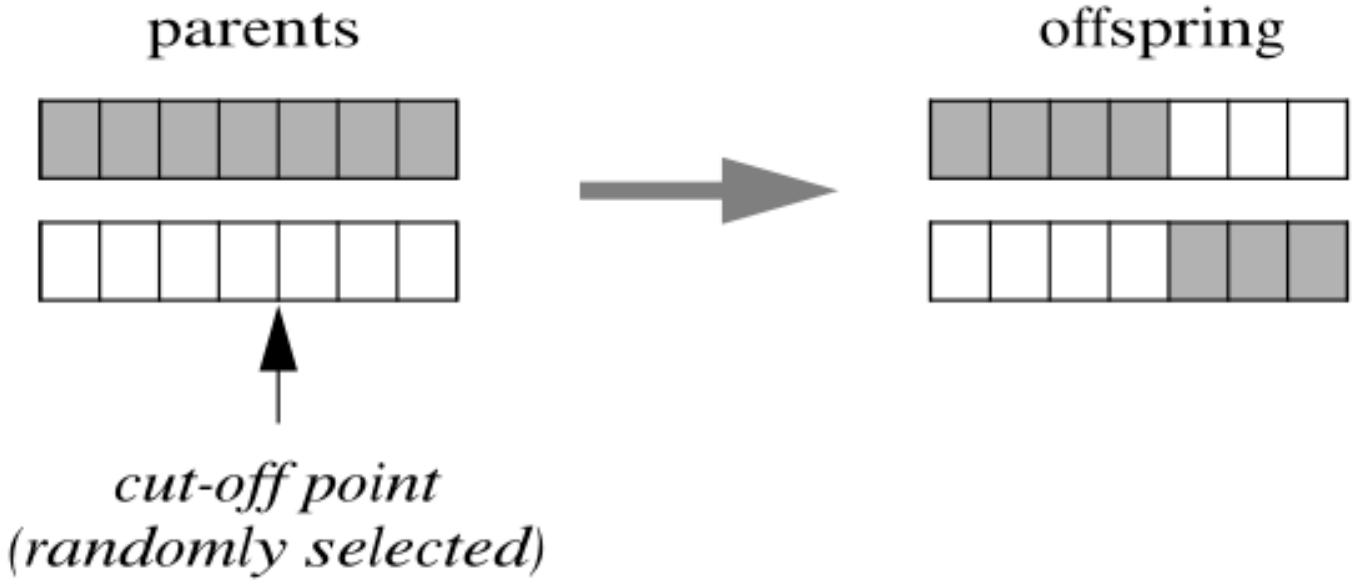
Values in GA (often 0/1's)
(allels in genetics)
values assigned to the corresponding
variables in the CSP

String in GA:
(chromosome in genetics)
compound labels in CSPs

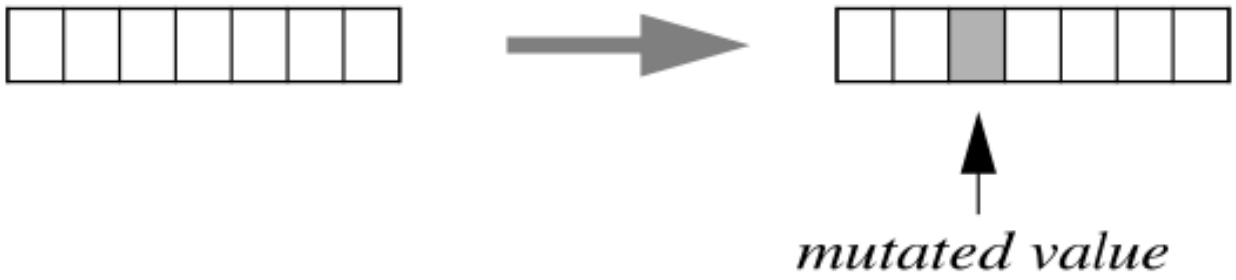


Building blocks in GA
(genes in genetics)
each building block represents
a variable in the CSP)

(a) Representation of candidate solutions of PCSPs in GA

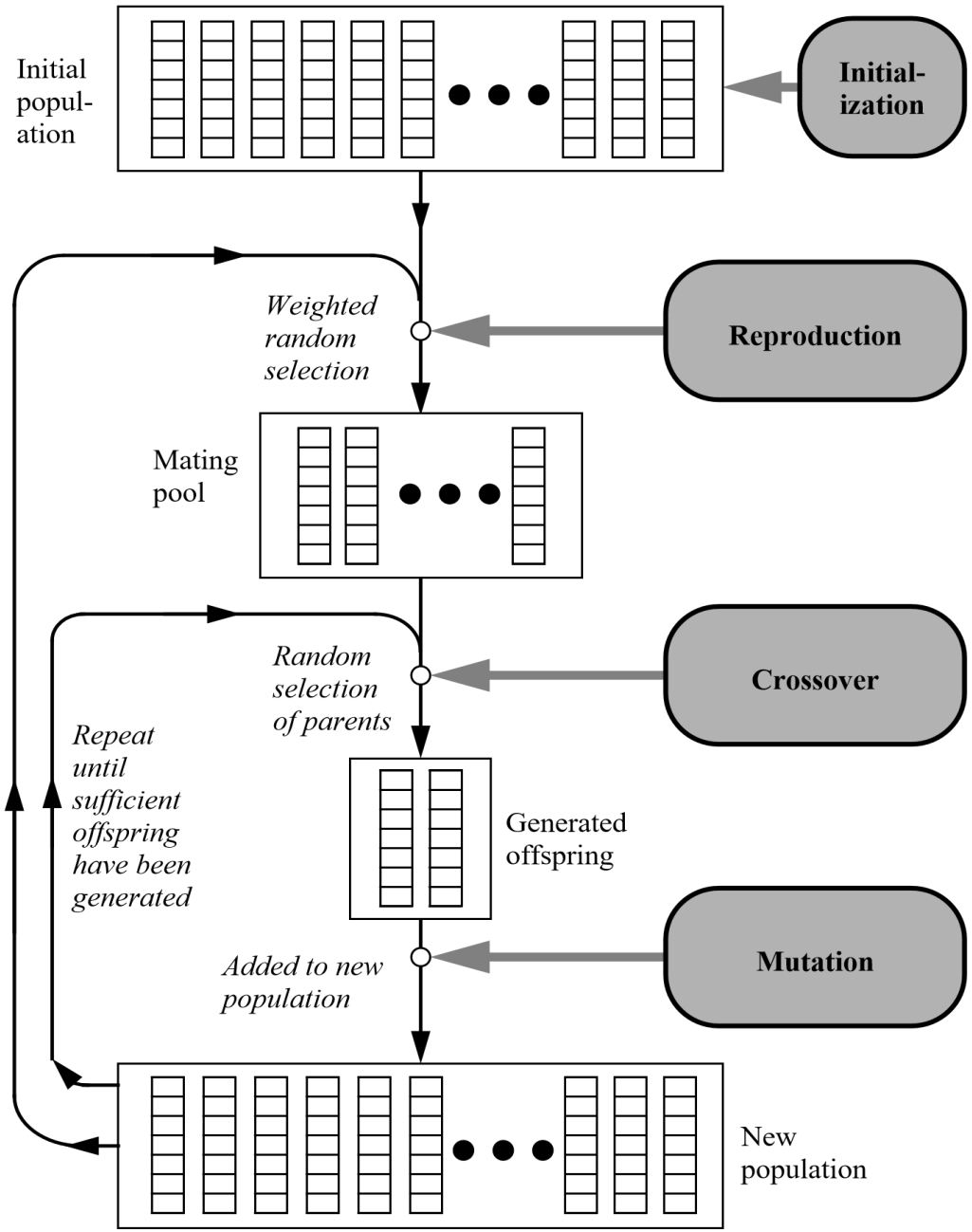


(b) Crossover — the building block of the parents are exchanged to form the new offspring



(c) Mutation — a random building block is picked, and its value changed

To apply GAs to optimization problems, a population of candidate solutions is generated and maintained. In a simple implementation, random strings could be generated in the initialization process. A more sophisticated initialization may ensure that all allels of all the building blocks are present in the initial population. The size of the population is one of the parameters of the GA which has to be set by the program designer. After the initial population is generated, the population is allowed to evolve dynamically.


Initial
popul-
ation
Initial-
ization
Weighted
random
selection
Reproduction
Mating
pool
Random
selection
of parents
Crossover
Repeat
until
sufficient
offspring
have been
generated
Generated
offspring
Added to new
population
Mutation
New
population

```
PROCEDURE CGA(f, PZ, ZMP, MutationRate)
/* f = the evaluation function; PZ = Population Size; ZMP = Size of the
    Mating Pool */
BEGIN
    Population ← Initialization(PZ);
    REPEAT
        Mating_Pool ← Reproduction(f, ZMP, Population);
        Population ← { };
        REPEAT
            Parent1 ← random element from Mating_Pool;
            Parent2 ← random element from Mating_Pool;
            Offspring ← Crossover(Parent1, Parent2);
            FOR each element os in Offspring DO
                IF (random number (between 0 and 1) ≤ MutationRate)
                THEN os ← Mutation(os);
            Population ← Population + Offspring;
        UNTIL size_of(Population) = PZ;
    UNTIL (converged(Population) ∨ resources_exhausted);
END /* of CGA */
```

Under a fixed set of GA operators, there are a number of parameters that one may set:

- the size of the population;
- the size of the mating pool;
- the frequency of mutation;
- the number of offspring to be generated from each pair of parents;
- the time the GA is allowed to run;
- the maximum number of iterations the GA is allowed to run;
- etc.

The effectiveness of a GA in an application, or in a particular problem, is dependent on a large number of factors, including the representation, the definition of the evaluation function and both the control and the operators and parameters used. Finding the right combination is the key to the success of a GA, hence the focus of much current research.

Applying GAs to CSOPs

For a CSOP with n variables, each string can be used to represent an n-compound tuple, where the building blocks represent the variables (in fixed order) each of which can take a value from a finite domain. Each schema in GA represents a compound label in a CSP.

What cannot be represented explicitly in a string are the constraints in a CSOP. To ensure that a GA generates legal compound labels (compound labels which satisfy the constraints), one may use one of the following strategies:

- make sure that the population contains only those strings which satisfy the constraints (by designing the initialization, crossover and mutation operators appropriately); or
- build into the evaluation function a *penalty function* which assigns low fitness values to strings that violate constraints. This effectively reduces the chance of those strings which violate certain constraints to reproduce.

10.3 The Partial Constraint Satisfaction Problem

Study of the *partial constraint satisfaction problem* (PCSP) is motivated by applications such as industrial scheduling, where one would normally like to utilize resources to their full. The constraints in a problem are often so tight that solutions are not available.

Often what a problem solver is supposed to do is find near solutions when the problem is over-constrained, so that it or its user will know how much the constraints should be relaxed. In other applications, the problem solver is allowed to violate some constraints at certain costs.

For example, shortage in manpower could sometimes be met by paying overtime or employing temporary staff; shortage in equipment could be met by hiring, leasing, etc. In these applications, one would often like to find near-solutions when the problem is over-constrained. We call such problems PCSPs.

Definition 10.2:
A **partial constraint satisfaction problem (PCSP)** is a quadruple:

$$(Z, D, C, g)$$

where (Z, D, C) is a CSP, and g is a function which maps every compound label to a numerical value, i.e. if cl is a compound label in the CSP then:

$$g: cl \rightarrow \text{numerical value}$$

Given a compound label cl, we call $g(cl)$ the g-value of cl. ■

The task in a PCSP is to find the compound label(s) with the optimal g-value with regard to some (possibly application-dependent) optimization function g.

The PCSP can be seen as a generalization of the CSOP defined above, since the set of solution tuples is a subset of the compound labels. In a maximization problem, a PCSP (Z, D, C, f) is equivalent to a CSOP (Z, D, C, g) where:

$$g(cl) = f(cl) \quad \text{if } cl \text{ is a solution tuple}$$

$$g(cl) = -\infty \quad \text{otherwise } (g(cl) = \infty \text{ in a minimization problem})$$

Important classes of PCSP and relevant techniques

PCSPs may take various forms, depending on their optimization functions (gs), and therefore it is difficult to name the relevant techniques. In the worst case, the whole search space must be searched because unlike in CSOPs, one cannot prune any part of the search space even if one can be sure that every compound label in it violates some constraints. Therefore, heuristics on the satisfiability of the problem become less useful for pruning off search spaces.

On the other hand, heuristics on the optimization function (i.e. estimation of the g-values) are useful for PCSPs. When such heuristics are available, the best known techniques for solving a PCSP is B&B.

We introduce two classes of PCSPs which are motivated by scheduling.

The minimal violation problem

In a CSP, if one is allowed to violate the constraints at some costs, then the CSP can be formalized as a PCSP where the optimization function g is one which maps every compound label to a non-positive numerical value. The task in such problems is to find n-compound labels (where n is the number of variables in the problem) which violate the minimum amount of constraints.

Definition 10.3:
A minimal violation problem (MVP) is a quadruple:

$$(Z, D, C, g)$$

where (Z, D, C) is a CSP, and g is a function which maps every compound label to a number:

$g(cl)$ = numerical value if cl is an n-compound label and $n = |Z|$
= infinity otherwise ■

The task in a MVP (Z, D, C, g) where $|Z| = n$ is to minimize $g(cl)$ for all n-compound labels cl.

MVPs can be found in scheduling, where constraints on resources can be relaxed (e.g. by acquiring additional resources) at certain costs. A function, call it c, maps every constraint to a relaxation cost:

$$c : C \rightarrow \text{numerical value}$$

The optimization function in the PCSP is then the sum of all the costs incurred in relaxing the violated constraints:

$$g(cl) = \sum_{C_S \in C \wedge \neg satisfies\,(cl,\, C_S)} c\,(C_S)$$

Another example of the MVP is graph matching where inexact matches are acceptable when exact matches do not exist. This will be the case when noise exists in the data, as would be the case in many real life applications. When the knowledge of the labels on the nodes and edges are unreliable, inexact matching will be acceptable.

Among the available techniques:

- branch and bound is applicable to the MVP when good heuristics are available;

- the heuristic repair method, which uses the min-conflict heuristic, is one relevant technique for solving the MVP; modify it to reflect the cost of the violation of the constraints

- the GENET approach which basically adds learning to a hill-climbing algorithm using the min-conflict heuristic, is also a good candidate to the MVP; initialize the weights of the connections to the negation of the costs of violating the corresponding constraint.

The maximal utility problem

In some applications, no constraint can be violated. When no solution tuple can be found, the problem solver would settle for k-compound labels (with k less than the total number of variables in the problem) which have the greatest "utility", where utility is user defined.

Definition 10.4:
A **maximal utility problem (MUP)** is a quadruple:

$$(Z, D, C, g)$$

where (Z, D, C) is a CSP, and g is a function which maps every compound label to a number:

$g(cl)$ = numerical value if the compound label cl violates no constraint
= minus infinity otherwise ■

The task in a MUP (Z, D, C, g) is to maximize $g(cl)$ for all k-compound labels cl.

MUPs can also be found in resource allocation in job-shop scheduling.

In some applications, one would like to assign resources to tasks, satisfying constraints which must not be violated.

For example, the capacity of certain machines cannot be exceeded; no extra manpower can be recruited to do certain jobs within the available time. A "utility", for example sales income, may be associated with the accomplishment of each job. If one cannot schedule to meet all the orders, one would like to meet the set of orders whose sum of utility is maximal. If all jobs have a uniform utility, then the task becomes "to finish as many jobs as possible".

Among the available techniques:

- the branch and bound algorithm is applicable to the MUP when heuristics are available;
- problem reduction is also applicable to MUPs; techniques such as forward checking and arc-consistency lookahead could be expected to search a smaller space;
- most of the variable ordering heuristics are applicable to problems where all solutions are required, and therefore are applicable to MUPs; the minimal width ordering (MWO) heuristic, the minimal bandwidth ordering (MBO) heuristic, and the fail first principle (FFP) attempt to improve search efficiency by different means.

To recapitulate, the MWO heuristic attempts to reduce the need to backtrack; the MBO heuristic attempts to reduce the distance of backtracking; the FFP attempts to prune off as much search space as possible when backtracking is required.

Heuristics on the g-values are useful for ordering the variables and their values in branch and bound. The efficiency of branch and bound is affected by the discovery of tight bounds at early stages.

Hill-climbing could be applicable to MUPs if appropriate heuristic functions (for guiding the direction of the climb) are available. In MUPs, one would like to hill-climb in the space of legal compound labels. Since the min-conflict heuristic aims at reducing the total number of constraints violated, it may not be very relevant to MUPs. Instead, one might need to use a heuristic which maximally increases the utility.

The GENET approach takes the min-conflict heuristic, and therefore needs modifications if it were to be used to tackle MUPs.